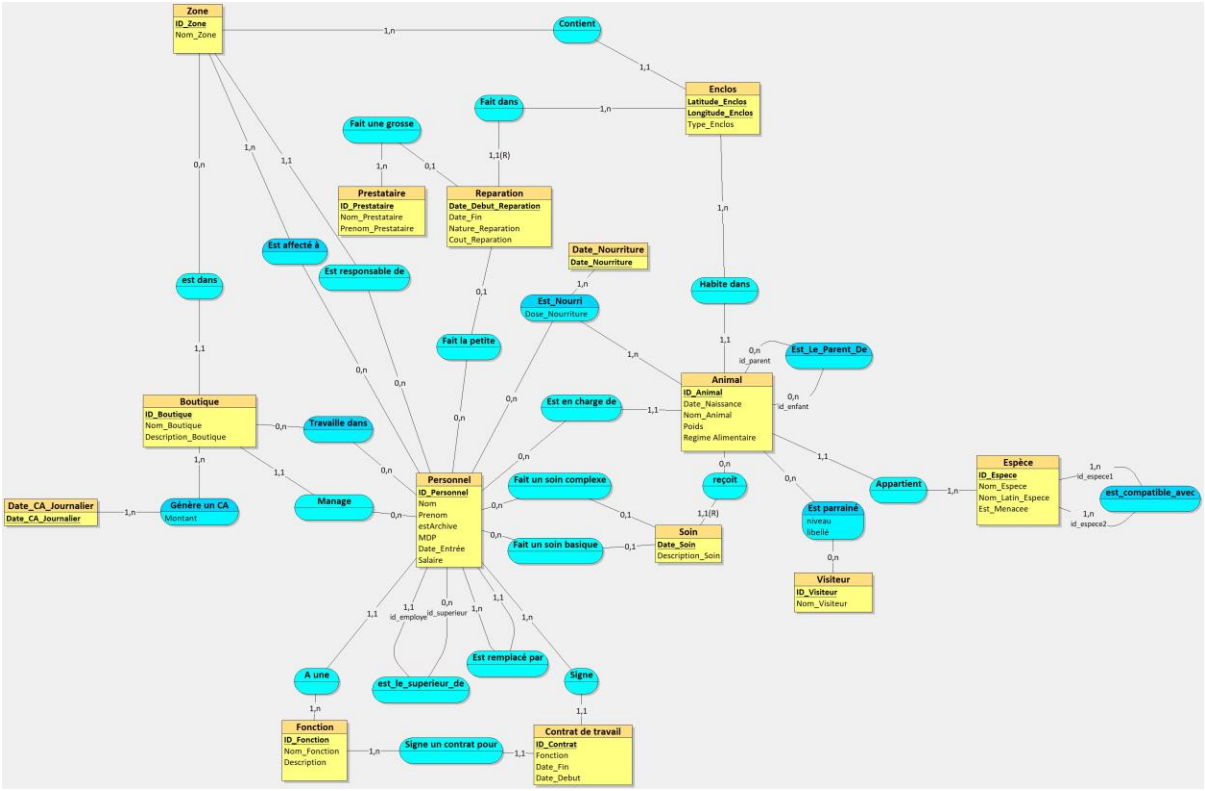
SAE Bases de données

Réalisée par :

DELIGNIERES Marius DELCROIX Lucas

1. Modèle Conceptuel des Données



2. Modèle Relationnel

Personnel(**ID_Personnel**, #ID_Remplacant, #ID_Superieur, #ID_Fonction, Nom, Prenom, Mail, MDP, Date_Entree, Salaire, Login, estArchive)

Contrat_Travail(**ID_Contrat**, #ID_Personnel, #ID_Fonction, Date_Debut, Date_Fin)

Boutique(**ID_Boutique**, #ID_Manager, #ID_Zone, Nom_Boutique, Description_Boutique)

Chiffre_Affaires(**#ID_Boutique, #Date_CA_Journalier**, Montant)

Prestataire(**ID_Prestataire**, Nom_Prestataire, Prenom_Prestataire)

Zone(**ID_Zone**, Nom_Zone, #ID_Manager)

Fonction(**ID_Fonction**, Nom_Fonction, Description)

Enclos(**Latitude_Enclos, Longitude_Enclos**, #ID_Zone, Type_Enclos)

Animal(**ID_Animal**, #(Latitude_Enclos,Longitude_Enclos), #ID_Espece,#ID_Soigneur_Atittré Date_Naissance, Nom_Animal, Poids, Regime_Alimentaire)

Visiteur(**ID_Visiteur**, Nom_Visiteur)

Espèce(**ID_Espece**,Nom_Espece,Nom_Latin_Espece,Est_Menacee)

Est_Parrainé(**#ID_Animal,#ID_Visiteur**, Niveau)

Est_Compatible_Avec(**#ID_Espece1,#ID_Espece2**)

Est_Le_Parent_De(**#ID_Parent,#ID_Enfant**)

Est_Nourri(**#ID_Animal, #Date_Nourriture**, #ID_Personnel, Dose_Nourriture)

Soin(**#ID_Animal, #ID_Personnel, #Date_Soin**, Description_Soin)

Est_Affectee_A(#**ID_Zone, #ID_Personnel**)

Travaille_Dans_La_Boutique(**#ID_Boutique, #ID_Personnel**)

Reparation(**Date_Debut_Reparation**, #(**Latitude_Enclos**,**Longitude_Enclos),** #ID_Personnel, #ID_Prestataire, Date_Fin, Nature_Reparation, Cout_Reparation)

3. Explications sur le MCD

Pour modéliser le parrainage entre un animal et un visiteur, l'idée derrière était simplement de permettre à un animal d'être parrainé par plusieurs personnes (ou personne, si personne n'aime cet animal...), d'où la cardinalité 0,n côté Animal et plusieurs personnes peuvent parrainer le même animal donc on met 0,n côté Visiteur également. D'autres solutions avaient été envisagées durant la conception mais elles étaient trop complexes et avaient des grosses failles (Une de nos solutions envisagées était qu'un visiteur pouvait parrainer plusieurs fois un même animal avec différents niveaux de parrainages, cela paraît étrange car un parrainage niveau "Or" est censé donné les prestations d'un niveau "Argent" et d'autres prestations supplémentaires. Mais ici on pouvait accumuler les mêmes prestations.)
Pour modéliser le lien de parenté entre deux animaux, on utilise ici une association réflexive entre Animal et Animal, avec une cardinalité de 0,n car un animal peut être ou non parent d'un ou plusieurs enfants. Un parent peut avoir plusieurs enfants et un enfant est l'enfant d'aucun parent dans le zoo ou de plusieurs parents (si le parent n'est pas dans le zoo, on l'ignore tout simplement)

Cette logique s'applique aussi du côté de l'entité Espèce et la gestion de la compatibilité entre espèces. Une espèce peut être compatible avec aucune autre espèce ou avec plusieurs espèces. D'où la cardinalité 0,n des deux côtés.

Pour modéliser la réparation des enclos par un prestataire ou un employé du zoo (en fonction du type de réparation), on a donc une entité Réparation qui est faible par rapport à Enclos (une réparation implique forcément un enclos donc sa clé primaire sera obligatoirement un identifiant de réparation suivi de l'identifiant de l'enclos). L'association entre Réparation et Prestataire est là pour modéliser le fait qu'un prestataire peut ou non (d'où la cardinalité 0,1) faire une réparation sur un Enclos, et différents prestataires peuvent faire une réparation (1,n) côté Prestataire).
Cette logique est identique entre Personnel et Réparation. Cela permet de modéliser le fait qu'un personnel ou un prestataire fait une réparation sur un enclos. Si un prestataire fait une réparation, alors c'est à 0 côté Personnel (la clé étrangère de Personnel vaudra donc null dans Réparation, ce qui est faisable car c'est une clé étrangère qui n'est pas clé primaire dans Réparation), et si c'est un personnel qui fait la réparation, alors c'est Prestataire qui est à NULL dans Réparation. (NB: c'est 0,n côté Personnel car un personnel n'est pas forcément du personnel d'entretien du zoo, là où un prestataire est forcément un prestataire pour l'entretien du zoo (d'où la cardinalité 1,n).)

La modélisation de la gestion de la nourriture d'un animal se fait grâce à une association maillée. Elle s'explique du fait qu'un membre du personnel qui est spécifiquement un soigneur s'occupe de nourrir l'animal (ce qui justifie la cardinalité 0,n côté Personnel car un personnel n'est pas forcément soigneur) et la gestion de la nourriture implique un ou plusieurs animaux dans le zoo (donc 1,n). Cependant, si on laisse cela tel quel, on dit que la gestion de la nourriture de l'animal se fait une fois par un soigneur, dès que tous les soigneurs se sont occupés de nourrir l'animal, on ne peut plus s'occuper de lui. Donc il faut une date en tant que pseudo-entité afin d'exprimer le fait que la gestion d'un animal est dépendante de l'animal, du soigneur et de la date du soin.
La gestion d'un soin pour un animal se fait grâce à l'entité Soin. En effet, il est demandé que plusieurs types de personnels puissent s'occuper du soin d'un animal. Le vétérinaire en cas de soins complexes et le soigneur attitré ou remplaçant en cas de soin basiques. Mais le fait de pouvoir savoir si c'est un vétérinaire ou non ne peut être géré au sein du MCD, il faut laisser la possibilité de pouvoir gérer ça lors de l'implémentation du site, d'où les deux associations "Fait un soin complexe" et "Fait un soin basique". Lors de l'implémentation de la base de données, le site devra être conçu de sorte que l'on puisse savoir si c'est un soigneur ou un vétérinaire qui a fait le soin. Si c'est l'un, l'autre est à NULL. L'implémentation du site devra également vérifier si le soigneur qui fait un soin est bien le soigneur attitré de l'animal, ou le remplaçant du soigneur. Et c'est pour cela que l'association "est en charge de" existe entre Personnel et Animal. Cependant lors de la création du site, une condition a bien été précisée afin de gérer le cas où un soigneur qui n'a pas un animal en charge essaye d'ajouter un soin.

Chaque membre du personnel a théoriquement la possibilité d'avoir un remplaçant. Cependant, lors de l'implémentation, cette fonctionnalité sera restreinte afin que seuls les soigneurs puissent avoir un remplaçant, comme demandé sur le sujet.
La hiérarchie au sein du zoo est modélisée par l'association réflexive "est_le_superieur_de" (le directeur a pour supérieur lui-même, sinon on met 0,1 et on met null). La hiérarchie du zoo est donc visible au sein de la base de données car chaque employé à un supérieur hiérarchique direct.

L'historique des employés est modélisé par l'entité "Contrat de travail" car chaque employé qui rentre dans le zoo possède un contrat de travail et si jamais un employé change de fonction au fil du temps, un autre contrat de travail sera signé et la traçabilité sera présente, il sera toujours possible de tracer les différents emplois d'un même employé au sein du zoo.
De plus, on peut imaginer la situation suivante: Si un employé quitte le zoo, son historique au sein de la base est-il toujours accessible ? La réponse est non, car il sera simplement supprimé de la base et toutes les données seront supprimées également. La solution à cela ? L'archivage. Un attribut estArchive est dans Personnel afin de savoir si oui ou non, l'utilisateur est toujours présent dans le zoo. C'est comme cela que l'on modélise le stockage des données sur une durée plus ou moins

longue et que l'on filtrera le personnel actif ou non au sein du site. Car il faut toujours avoir une traçabilité d'un ancien employé (Selon la loi en tout cas)

Le chiffre d'affaires d'une boutique n'est pas fixe, on ne peut donc pas directement le stocker dans l'entité Boutique, il a donc été nécessaire d'en faire une entité afin de stocker dynamiquement les données au fil du temps, afin d'avoir une entité pour contenir tous les chiffres d'affaires générés par l'intégralité des boutiques et d'avoir une traçabilité concernant le chiffre d'affaires mensuel pour chaque boutique ainsi que pour l'intégralité du zoo. Pour cela, on a dû modéliser une pseudo-entité afin d'avoir la date et l'identifiant de la boutique dans notre association "Génère un CA", car un chiffre d'affaires quotidien est dépendant de la date et de la boutique qui a généré ce chiffre d'affaires.

4. Dictionnaire de données

Attribut	Description	Type	Longueur	Contraintes	Règle de calcul
ID_Personnel	Identifiant du membre du personnel	Numérique		Clé	
Nom	Nom du membre du personnel	Chaine de caractères	50		
Prenom	Prénom du membre du personnel	Chaine de caractères	50		
estArchive	Représente si le membre du personnel est parti ou non	Chaine de caractères			
Login	Identifiant de connexion de l'utilisateur	Chaîne de caractère	101		
MDP	Mot de passe hashé du personnel (bcrypt)	Chaine de caractères	255		
Date_Entree	Date d'entrée du membre du personnel	Date			Date_Entree <= SYSDATE
Salaire	Salaire de la personne	Nombre à virgules (2 chiffres)			
ID_Fonction	Identifiant d'une fonction au sein du zoo	Numérique		Clé	
Nom_Fonction	Nom d'une fonction au sein du zoo	Chaine de	50		

		caractères			
Description	Principales activités d'une fonction au sein du zoo	Chaine de caractères	200		
ID_Contrat	Identifiant d'un contrat de travail	Numérique			
Date_Fin	Date de la fin du contrat	Date			Date_Fin > Date_Debut
Date_Debut	Date du début du contrat	Date			Date_Debut < Date_Fin
ID_Animal	Identifiant d'un animal dans le zoo	Numérique		Clé	
Date_Naissance	Date de naissance de l'animal du zoo	Date			Date_Naissance < SYSDATE
Nom_Animal	Nom de l'animal	Chaine de caractères	50		
Poids	Poids de l'animal	Nombre à virgules (2 chiffres)			Poids > 0
Regime_Alimentaire	Régime alimentaire de l'animal	Type énuméré			'Carnivore' ou 'Herbivore' ou 'Omnivore'
ID_Visiteur	Identifiant d'un visiteur qui parraine au sein du zoo	Numérique		Clé	
Nom_Visiteur	Nom du visiteur	Chaine de caractères	50		
Niveau	Niveau de parrainage	Type énuméré			'Or' ou 'Argent' ou 'Bronze'
ID_Espèce	Identifiant d'une espèce présente dans le zoo	Numérique		Clé	
Nom_Espece	Nom d'une espèce présente dans le zoo	Chaine de caractères	100		

Nom_Latin_Espece	Nom latin d'une espèce présente dans le zoo	Chaine de caractères	100		
Est_Menacee	Représente si cette espèce est menacée ou non	Type énuméré			0 ou 1
Date_Soin	Date d'un soin sur un animal	Date		Clé	
Description_Soin	Description du soin prodigué sur l'animal	Chaine de caractères	200		
Date_Nourriture	Date à laquelle un soigneur a donné de la nourriture à un animal	Date			Date_Nourriture <= SYSDATE
Dose_Nourriture	Dose de nourriture donnée à l'animal	Date			Dose_Nourriture > 0
ID_Prestataire	Identifiant d'un prestataire	Numérique		Clé	
Nom_Prestataire	Nom d'un prestataire	Chaine de caractères	50		
Prenom_Prestataire	Prénom d'un prestataire	Chaine de caractères	50		
Latitude_Enclos	Latitude d'un enclos au sein du zoo	Numérique		Clé	
Longitude_Enclos	Longitude d'un enclos au sein du zoo	Numérique		Clé	
Type_Enclos	Type de l'enclos	Chaine de caractères	50		
Date_Debut_Reparation	Date de début d'une réparation sur un enclos	Date		Clé	Format 'JJ-DD-YYYY' Date_Debut_Reparation < Date_Fin
Date_Fin	Date de la fin de réparation	Date			Date_Fin > Date_Debut_Reparation

Nature_Reparation	Description de la réparation	Chaine de caractères	100		
Cout_Reparation	Coût total de la réparation	Nombre à virgules (2 chiffres)			Cout_Reparation > 0
ID_Zone	Identifiant d'une zone au sein du zoo	Numérique		Clé	
Nom_Zone	Nom d'une zone	Chaine de caractères	100		
ID_Boutique	Identifiant d'une boutique au sein du zoo	Numérique		Clé	
Nom_Boutique	Nom d'une boutique	Chaine de caractères	100		
Description_Boutique	Description de ce que vend une boutique	Chaine de caractères	200		
Montant	Montant généré par une boutique sur une journée	Nombre à virgules (2 chiffres)			Montant >= 0
Date_CA_Journalier	Date à laquelle le montant a été généré	Date		Clé	Date_CA_Journalier <= SYSDATE

5. Requêtes SQL diverses et variées

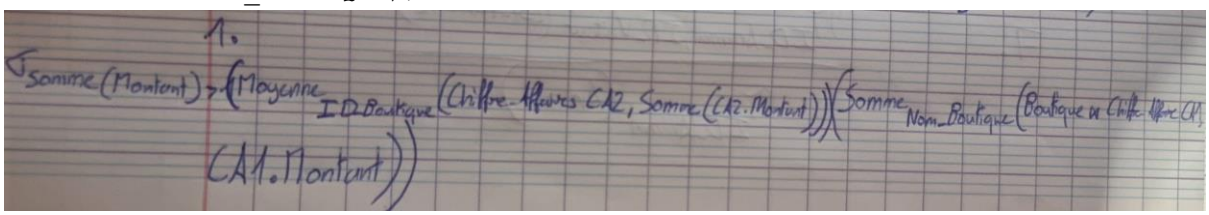
Les requêtes SQL conçues pour cette partie ont été réalisées dans le but de répondre à l'intégralité des questions proposées ci-dessous :

1. Quelles sont les boutiques dont le chiffre d'affaires total dépasse la moyenne de toutes les boutiques ?
2. Pour chaque zone, donner le chiffre d'affaires total de ses boutiques.
3. Lister le nombre de soins faits pour tous les soigneurs.
4. Pour chaque espèce, donner le poids moyen des animaux, uniquement pour les espèces ayant au moins 3 animaux.
5. Afficher les animaux dont le poids est supérieur au poids moyen de tous les animaux

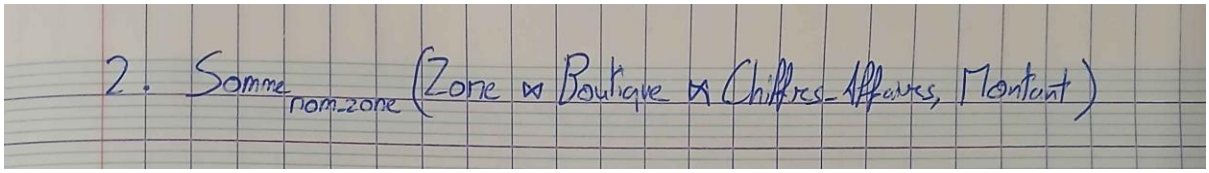
6. Quels employés gagnent plus que leur supérieur hiérarchique ?
7. Quels visiteurs ont parrainé tous les animaux ?
8. Liste des personnes qui sont soit managers de zone, soit managers de boutique dans des boutiques existantes.
9. Quelles espèces ont des animaux dans plusieurs zones différentes, et combien y-a-t-il d'animaux de cette espèce ?
10. Quels visiteurs parrainent des animaux de toutes les espèces menacées ?
11. Afficher tous les employés fictifs, c'est à dire tous les employés qui n'ont fait ni soin, ni réparation et qui ne sont pas managers de boutique ou de zone.
12. Créer une vue permettant d'afficher la liste des animaux avec leur espèce et leur soigneur responsable.

Réponses :

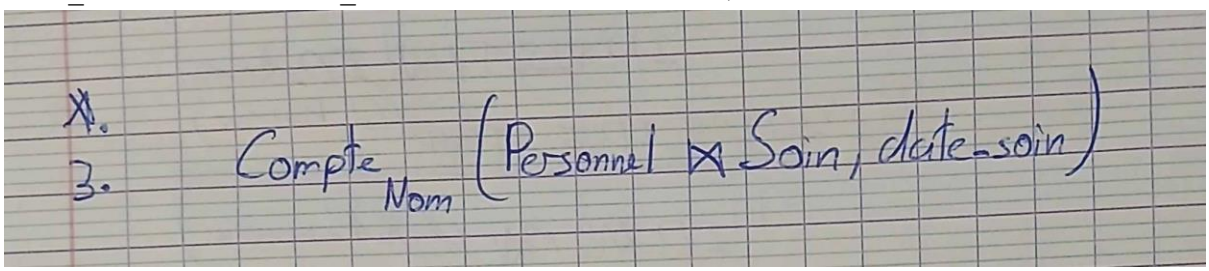
1.
```
SELECT B.NOM_BOUTIQUE, SUM(CA1.MONTANT) FROM BOUTIQUE B, CHIFFRE_AFFAIRES
CA1 WHERE B.ID_BOUTIQUE = CA1.ID_BOUTIQUE GROUP BY B.NOM_BOUTIQUE HAVING
SUM(CA1.MONTANT) > (SELECT AVG(SUM(CA2.MONTANT)) FROM CHIFFRE_AFFAIRES CA2
GROUP BY CA2.ID_BOUTIQUE);
```



2.
```
SELECT NOM_ZONE, SUM(CA.MONTANT) FROM ZONE Z, BOUTIQUE B, Chiffre_Affaires
CA WHERE Z.ID_ZONE = B.ID_ZONE AND B.ID_BOUTIQUE = CA.ID_BOUTIQUE GROUP BY
Z.NOM_ZONE ORDER BY Z.NOM_ZONE;
```



3.
```
SELECT P.NOM, COUNT(S.DATE_SOIN) FROM PERSONNEL P, SOIN S WHERE
P.ID_PERSONNEL = S.ID_SOIGNEUR GROUP BY P.NOM;
```



4.
```
SELECT E.NOM_ESPECE, AVG(A.POIDS) FROM ESPECE E, ANIMAL A WHERE E.ID_ESPECE
= A.ID_ESPECE GROUP BY E.NOM_ESPECE HAVING COUNT(A.ID_ANIMAL) >= 3;
```

4. $\sigma_{Compte(ID_Animal) \geq 3}\left(\text{Moyenne}_{Nom_Espece}\left(Espece \bowtie Animal, Poids\right)\right)$

5.
```
SELECT A.NOM_ANIMAL, A.POIDS FROM ANIMAL A WHERE A.POIDS > (SELECT
AVG(POIDS)FROM ANIMAL);
```

5. $\pi_{NOM_ANIMAL, poids}\left(\sigma_{poids > (\pi_{MOYENNE(Animal, poids)})}\right)(Animal)$

6.
```
SELECT P1.NOM FROM PERSONNEL P1 JOIN PERSONNEL P2 ON P1.ID_SUPERIEUR =
P2.ID_PERSONNEL AND P1.SALAIRE > P2.SALAIRE;
```

7.

<u>Version 1:</u>
```
SELECT EP.ID_VISITEUR FROM EST_PARRAINE EP GROUP BY EP.ID_VISITEUR HAVING
COUNT(DISTINCT EP.ID_ANIMAL) = (SELECT COUNT(*) FROM ANIMAL);
```

<u>Version 2:</u>
```
SELECT V.NOM_VISITEUR FROM VISITEUR V WHERE NOT EXISTS (SELECT A.ID_ANIMAL
FROM ANIMAL A WHERE NOT EXISTS (SELECT EP.ID_ANIMAL FROM EST_PARRAINE EP
WHERE V.ID_VISITEUR = EP.ID_VISITEUR AND EP.ID_ANIMAL = A.ID_ANIMAL));
```

<u>Version 3:</u>
```
SELECT V.NOM_VISITEUR FROM VISITEUR V WHERE NOT EXISTS (SELECT A.ID_ANIMAL
FROM ANIMAL A MINUS (SELECT EP.ID_ANIMAL FROM EST_PARRAINE EP WHERE
EP.ID_VISITEUR = V.ID_VISITEUR));
```

7. $\dfrac{\pi_{ID_Animal, ID_Visiteur}(Est_Parraine)}{\pi_{ID_Animal}(Animal)}$

8.
```
SELECT P.NOM FROM PERSONNEL P JOIN BOUTIQUE B ON P.ID_PERSONNEL =
B.ID_MANAGER UNION (SELECT P.NOM FROM PERSONNEL P JOIN ZONE Z ON
P.ID_PERSONNEL = Z.ID_MANAGER);
```

8. $\pi_{Nom}(Personnel \bowtie Boutique) \cup \pi_{Nom}(Personnel \bowtie Zone)$

9.
```
SELECT ES.NOM_ESPECE, COUNT(*) FROM ESPECE ES JOIN ANIMAL A ON ES.ID_ESPECE
= A.ID_ESPECE JOIN ENCLOS E ON A.LONGITUDE_ENCLOS = E.LONGITUDE AND
A.LATITUDE_ENCLOS = E.LATITUDE GROUP BY ES.NOM_ESPECE HAVING COUNT(DISTINCT
E.ID_ZONE) > 1;
```

9. $\sigma_{Compte(ID_Zone) > 1}\left(Compte_{Nom_Espece}\left(Espece \bowtie Animal \bowtie_{latitude_enclos = latitude \wedge longitude_enclos = longitude} Enclos\right)\right)$

10.

<u>Version 1:</u>
```
SELECT V.NOM_VISITEUR FROM EST_PARRAINE EP JOIN ANIMAL A ON EP.ID_ANIMAL =
A.ID_ANIMAL JOIN ESPECE E ON E.ID_ESPECE = A.ID_ESPECE JOIN VISITEUR V ON
EP.ID_VISITEUR = V.ID_VISITEUR AND E.Est_Menacee = 1 GROUP BY
V.NOM_VISITEUR HAVING COUNT(DISTINCT E.ID_ESPECE) = (SELECT COUNT(*) FROM
ESPECE WHERE EST_MENACEE=1);
```

<u>Version 2:</u>
```
SELECT V.NOM_VISITEUR FROM VISITEUR V WHERE NOT EXISTS (SELECT E.ID_ESPECE
FROM ESPECE E WHERE E.EST_MENACEE = 1 AND NOT EXISTS (SELECT EP.ID_ANIMAL
FROM EST_PARRAINE EP JOIN ANIMAL A ON EP.ID_ANIMAL = A.ID_ANIMAL WHERE
A.ID_ESPECE = E.ID_ESPECE AND EP.ID_VISITEUR = V.ID_VISITEUR));
```

<u>Version 3:</u>
```
SELECT V.NOM_VISITEUR FROM VISITEUR V WHERE NOT EXISTS (SELECT E.ID_ESPECE
FROM ESPECE E WHERE E.EST_MENACEE = 1 MINUS (SELECT E.ID_ESPECE FROM ESPECE
E JOIN ANIMAL A ON E.ID_ESPECE = A.ID_ESPECE JOIN EST_PARRAINE EP ON
A.ID_ANIMAL = EP.ID_ANIMAL AND EP.ID_VISITEUR = V.ID_VISITEUR));
```

10. $\dfrac{\pi_{ID_Animal, Nom_Visiteur}\left(\sigma_{Est_Menacee = 1}\left(Espece \bowtie Animal \bowtie Est_Parraine \bowtie Visiteur\right)\right)}{\pi_{ID_Animal}\left(\sigma_{Est_Menacee = 1}\left(Animal \bowtie Espece\right)\right)}$

11.
```
FROM Personnel MINUS
    (
            SELECT ID_Soigneur AS ID_Personnel, P.Nom, P.Prenom FROM Soin S
JOIN Personnel P ON P.ID_Personnel = S.ID_Soigneur
        UNION
```

```
            SELECT ID_Veterinaire AS ID_Personnel, P.Nom, P.Prenom FROM
Soin S JOIN Personnel P ON P.ID_Personnel = S.ID_Veterinaire
        UNION
            SELECT R.ID_Personnel, P.Nom, P.Prenom FROM Reparation R JOIN
Personnel P ON P.ID_Personnel = R.ID_Personnel
        UNION
            SELECT Z.ID_Manager, P.Nom, P.Prenom FROM Zone Z JOIN Personnel
P ON P.ID_Personnel = Z.ID_Manager
        UNION
            SELECT B.ID_Manager, P.Nom, P.Prenom FROM Boutique B JOIN
Personnel P ON P.ID_Personnel = B.ID_Manager
        UNION
            SELECT TDB.ID_Personnel, P.Nom, P.Prenom FROM
Travaille_Dans_La_Boutique TDB JOIN Personnel P ON TDB.ID_Personnel =
P.ID_Personnel
    );
```

11. $\pi_{ID_Personnel, Nom, Prenom}(Personnel - (\pi_{ID_Soigneur}(Soin) \cup \pi_{ID_Veterinaire}(Soin) \cup \pi_{ID_Personnel}(Reparation) \cup \pi_{ID_Manager}(Zone) \cup \pi_{ID_Manager}(Boutique) \cup \pi_{ID_Personne}(Travaille_Dans_La_Boutique)))$

12.

```
CREATE VIEW VueAnimauxEspeceSoigneur AS SELECT A.ID_Animal, A.Nom_Animal,
E.Nom_Espece,P.Nom AS Nom_Soigneur, P.Prenom AS Prenom_Soigneur FROM Animal
A JOIN Espece E  A.ID_Espece = E.ID_Espece JOIN Personnel P ON
A.ID_Soigneur = P.ID_Personnel;
```